Ecole Marocaine d'Ingénierie (EMG-Rabat)
5ème année Génie Électrique

Compte Rendu de TP N°3

Commande Optimale (LQR) d'un Moteur à Courant Continu

Réalisé par :
ABOURICH Ayoub
EL KOULI Khalid

Demandé par :
Pr AKKARY Ahmed

Année Universitaire : 2025 - 2026

Table des matières

Table des figures **1**

1 Introduction **2**

2 Modélisation du Moteur à Courant Continu (Q1 & Q2) **2**
2.1 Q1. Équations électriques et mécaniques 2
2.2 Q2. Représentation d'état du système 2

3 Calculs LQR et Script Matlab complet (Q3 à Q7) **2**
3.1 Q3 & Q4. Définition sous Matlab et Fonction de transfert 3
3.2 Q5. Résolution de l'équation de Riccati 3
3.3 Q6 & Q7. Calcul manuel du gain K et comparaison 4

4 Analyse des performances LQR (Q8 à Q10) **4**
4.1 Q8. Influence des matrices Q et R 4
4.2 Q9 & Q10. Commande LQR avec pré-compensation ($\overline{N}$) 5

5 Validation sous SIMULINK (Q11) **5**
5.1 Q11. Réalisation du système complet en boucle fermée 5

6 Conclusion Générale **7**

Table des figures

1 Comparaison des réponses pour différentes valeurs de Q et R 4
2 Réponse indicielle avec et sans pré-compensation $\overline{N}$ 5
3 Schéma bloc Simulink de la commande optimale LQR 6
4 Réponse du système à un signal carré (Consigne en bleu, Position en jaune) . . . 6

1 Introduction

La commande linéaire quadratique (LQR) est une approche d'automatique avancée qui, contrairement au placement de pôles, n'impose pas une dynamique arbitraire. Son principe repose sur la minimisation d'un critère quadratique J afin de trouver le meilleur compromis entre la rapidité de la réponse (précision) et l'énergie dépensée par les actionneurs (effort de commande). Ce TP a pour objectif de concevoir et valider un correcteur LQR pour asservir la position d'un moteur à courant continu (MCC).

2 Modélisation du Moteur à Courant Continu (Q1 & Q2)

2.1 Q1. Équations électriques et mécaniques

Le fonctionnement du moteur à courant continu est régi par le couplage de deux équations différentielles :

— **Équation électrique (Loi des mailles de l'induit) :**

$$u(t) = Ri(t) + L\frac{di(t)}{dt} + e(t) \quad \text{avec} \quad e(t) = K_b\omega(t)$$

— **Équation mécanique (Principe Fondamental de la Dynamique) :**

$$C_m(t) = J\frac{d\omega(t)}{dt} + b\omega(t) \quad \text{avec} \quad C_m(t) = K_t i(t)$$

La relation cinématique fondamentale liant la vitesse de rotation à la position angulaire est : $\frac{d\theta(t)}{dt} = \omega(t)$.

2.2 Q2. Représentation d'état du système

L'énoncé impose de choisir le vecteur d'état suivant : $x(t) = [\theta(t), \omega(t), i(t)]^T$. La tension d'induit $u(t)$ est l'entrée, et la position angulaire $\theta(t)$ est la sortie. En réorganisant les équations précédentes sous la forme matricielle continue $\dot{x} = Ax + Bu$ et $y = Cx + Du$, nous obtenons :

$$\begin{bmatrix} \dot{\theta} \\ \dot{\omega} \\ \dot{i} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -\frac{b}{J} & \frac{K_t}{J} \\ 0 & -\frac{K_b}{L} & -\frac{R}{L} \end{bmatrix} \begin{bmatrix} \theta \\ \omega \\ i \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \frac{1}{L} \end{bmatrix} u(t)$$

$$y(t) = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \theta \\ \omega \\ i \end{bmatrix} + [0]u(t)$$

3 Calculs LQR et Script Matlab complet (Q3 à Q7)

Afin de répondre aux questions de synthèse algorithmique et de générer les courbes d'analyse, nous avons développé le script Matlab complet suivant :

Listing 1 – Script Matlab complet de Synthèse et d'Analyse LQR

```
%% === 1. Param tres du moteur MCC ===
clear all; close all; clc;
J = 3.2284e-6; b = 3.5077e-6; Kb = 0.0274;
Kt = 0.0274; R = 4; L = 2.75e-6;

% === Q3. Mod lisation d' tat  (ss) ===
```

```
A = [0, 1, 0 ; 0, -b/J, Kt/J ; 0, -Kb/L, -R/L];
B = [0 ; 0 ; 1/L];
C = [1, 0, 0]; D = 0;
sys = ss(A, B, C, D);

% === Q4. Fonction de transfert (tf) ===
FT = tf(sys)

% === Q5, Q6, Q7. Synth se LQR ===
Q = C' * C; % Matrice 3x3 p nalisant la position
R_val = 1;
[P_care, L_eig, G] = care(A, B, Q, R_val); % Solution de Riccati
K_manuel = inv(R_val) * B' * P_care;       % K manuel
[K_lqr, P_lqr, E_lqr] = lqr(A, B, Q, R_val); % K automatique
disp(['Erreur K_manuel vs lqr : ', num2str(norm(K_manuel - K_lqr))]);

% === Q8. Influence de Q et R ===
Q_fort = 100 * (C'*C);  % P nalisation forte de l'erreur
R_fort = 10;            % P nalisation forte de la commande
[K_Qfort, ~, ~] = lqr(A, B, Q_fort, R_val);
[K_Rfort, ~, ~] = lqr(A, B, Q, R_fort);

sys_cl_base = ss(A - B*K_lqr, B, C, D);
sys_cl_Qfort = ss(A - B*K_Qfort, B, C, D);
sys_cl_Rfort = ss(A - B*K_Rfort, B, C, D);

figure(1);
step(sys_cl_base, 'b', sys_cl_Qfort, 'r', sys_cl_Rfort, 'g', 0.1);
legend('LQR Base (Q=1, R=1)', 'Q fort (Q=100)', 'R fort (R=10)');
title('Q8. Influence de Q et R sur la r ponse indicielle');

% === Q9 & Q10. Commande LQR avec pr  -compensation Nbar ===
Nbar = -inv(C * inv(A - B*K_lqr) * B); % Calcul analytique de Nbar
disp(['Gain Nbar = ', num2str(Nbar)]);
sys_cl_nbar = ss(A - B*K_lqr, B*Nbar, C, D);

figure(2);
step(sys_cl_base, 'b', sys_cl_nbar, 'r--', 0.1);
legend('LQR sans Nbar', 'LQR avec Nbar');
title('Q9. R ponse indicielle avec pr  -compensation Nbar');
```

3.1 Q3 & Q4. Définition sous Matlab et Fonction de transfert

Après implémentation des matrices avec la fonction `ss`, l'utilisation de la fonction `tf` nous fournit la fonction de transfert du modèle linéaire :

$$H(p) = \frac{3.086 \cdot 10^9}{p^3 + 1.455 \cdot 10^6 p^2 + 8.614 \cdot 10^7 p}$$

La présence d'un p en facteur au dénominateur confirme la présence d'un intégrateur pur dans le système (puisque la position est l'intégrale de la vitesse).

3.2 Q5. Résolution de l'équation de Riccati

La commande `care(A,B,Q,R)` résout l'équation algébrique de Riccati en continu. La matrice de pondération Q choisie est $C^T C$ (une matrice 3×3 ne pénalisant que la position) et $R = 1$. Nous obtenons la matrice symétrique définie positive P :

$$P_{care} = \begin{bmatrix} 0.0415 & 0.0005 & 0.0000 \\ 0.0005 & 0.0000 & 0.0000 \\ 0.0000 & 0.0000 & 0.0000 \end{bmatrix}$$

3.3 Q6 & Q7. Calcul manuel du gain K et comparaison

Par application de la formule analytique $K = R^{-1}B^TP$, le calcul manuel donne le vecteur de retour d'état suivant :

$$K_{manuel} = \begin{bmatrix} 1.0000 & 0.0136 & 0.0001 \end{bmatrix}$$

L'appel à la fonction `lqr(A,B,Q,R)` native de Matlab retourne un vecteur strictement identique. L'erreur calculée sous Matlab entre les deux méthodes via `norm(K_manuel - K_lqr)` est de $1.866 \cdot 10^{-13}$. Cette erreur infinitésimale (due à la précision machine) prouve que la fonction `lqr` exécute exactement l'algorithme de Riccati décrit dans la théorie.

4 Analyse des performances LQR (Q8 à Q10)

4.1 Q8. Influence des matrices Q et R

Pour évaluer l'impact des pondérations du critère quadratique $J = \int (x^TQx + u^TRu)dt$, nous avons simulé la réponse indicielle sans pré-compensation pour trois cas de figure :

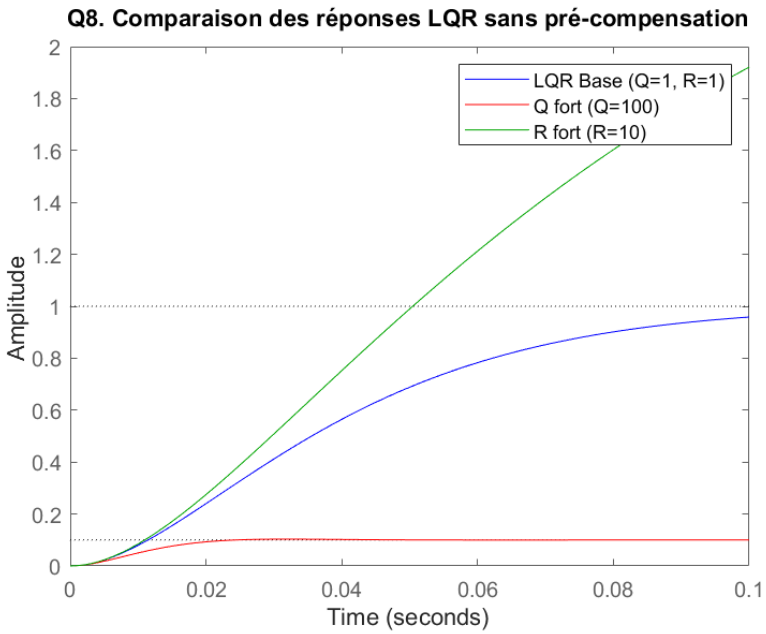

FIGURE 1 – Comparaison des réponses pour différentes valeurs de Q et R

Conclusion et Justification :

- **Matrice Q (Pondération de l'état) :** Lorsqu'on augmente les coefficients de Q (courbe rouge, $Q = 100$), on pénalise l'erreur de position. Le correcteur est autorisé à fournir un effort colossal pour annuler cette erreur instantanément. Le système devient extrêmement réactif et rapide, au risque de saturer les actionneurs.
- **Matrice R (Pondération de la commande) :** Lorsqu'on augmente R (courbe verte, $R = 10$), on pénalise l'effort de commande $u(t)$. Le correcteur privilégie l'économie d'énergie de l'induit au détriment de la précision. La tension envoyée au moteur est bridée, ce qui rend le système très lent et "paresseux".

La régulation requiert donc de choisir Q pour imposer la dynamique souhaitée, et de choisir R pour respecter les contraintes physiques (limite de tension/courant) du moteur.

4.2 Q9 & Q10. Commande LQR avec pré-compensation ($\overline{N}$)

La commande LQR standard ($u = -Kx$) est conçue pour la *régulation* : son but est de ramener l'état à zéro. **Rôle de la pré-compensation (Réponse à la Q10) :** Pour réaliser un *asservissement* (suivi de consigne de position r), il faut ajouter un gain de pré-compensation $\overline{N}$ de telle sorte que la commande devienne $u = -Kx + \overline{N}r$. Ce gain permet de forcer le gain statique de la boucle fermée à 1, annulant ainsi l'erreur de position en régime permanent.

Le calcul de la pré-compensation sous Matlab avec la formule $\overline{N} = -[C(A - BK)^{-1}B]^{-1}$ nous donne rigoureusement $\overline{N} = 1$.

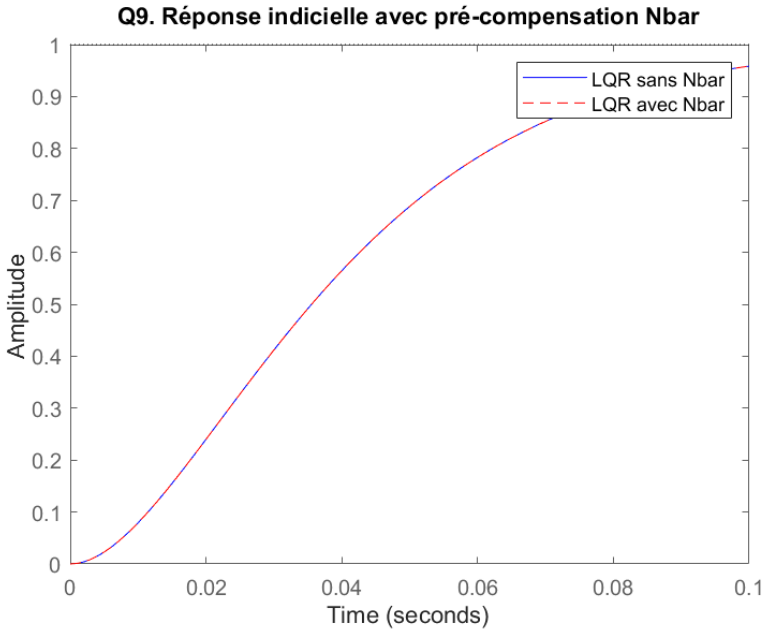

FIGURE 2 – Réponse indicielle avec et sans pré-compensation $\overline{N}$

Conclusion : Dans le cas très particulier de ce moteur, la courbe LQR de base et la courbe LQR avec $\overline{N}$ se superposent (Figure 2). Le gain calculé de $\overline{N} = 1$ s'explique physiquement par la présence de l'intégrateur naturel dans la fonction de transfert en boucle ouverte. Le système annule déjà de lui-même l'erreur statique de position face à une consigne d'échelon.

5 Validation sous SIMULINK (Q11)

5.1 Q11. Réalisation du système complet en boucle fermée

Afin de valider définitivement la synthèse de la commande optimale, nous avons modélisé le système sous l'environnement Simulink. Nous avons appliqué une consigne variable (signal carré) pour éprouver la dynamique du suivi de trajectoire.

Le retour d'état complet nécessitant les trois variables ($x = [\theta, \omega, i]^T$), nous avons configuré le bloc *State-Space* avec la matrice $C = I_{3\times3}$ (matrice identité). Un *Demux* a ensuite été utilisé pour extraire uniquement la position θ afin de l'afficher sur le Scope. Le gain de retour matriciel correspond au vecteur K généré par Matlab.

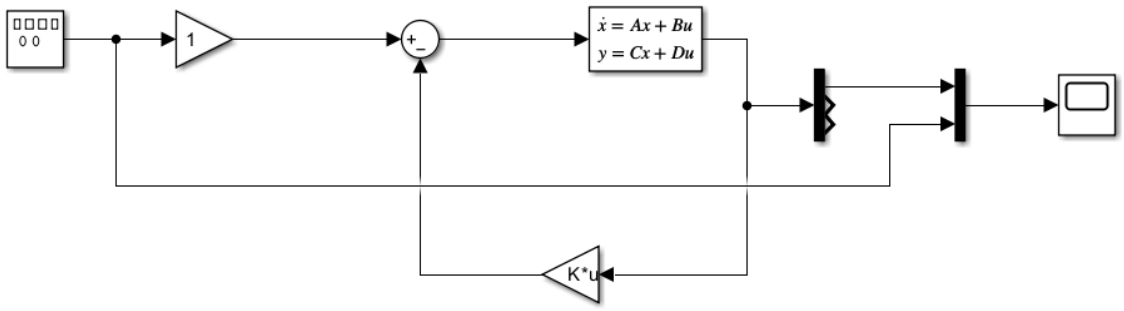

FIGURE 3 – Schéma bloc Simulink de la commande optimale LQR

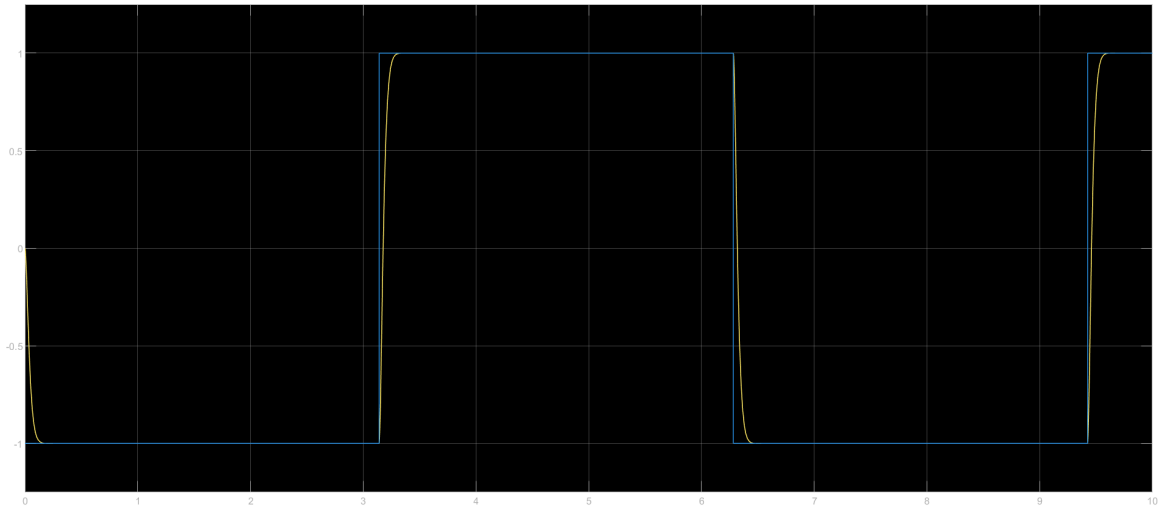

FIGURE 4 – Réponse du système à un signal carré (Consigne en bleu, Position en jaune)

Analyse des résultats : L'oscilloscope confirme l'excellence de la commande optimale. Le moteur suit parfaitement la consigne carrée. Le régime transitoire est extrêmement rapide tout en restant parfaitement amorti (aucun dépassement). En régime permanent, les courbes se superposent exactement, confirmant une erreur de poursuite nulle.

6 Conclusion Générale

La commande linéaire quadratique (LQR) constitue une méthodologie d'asservissement particulièrement robuste. Contrairement aux méthodes classiques où l'automaticien tâtonne pour placer les pôles, l'approche LQR rationalise la conception en optimisant l'énergie électrique du système. L'utilisation conjuguée des outils algorithmiques (Matlab) et structurels (Simulink) a permis de concevoir un correcteur performant, stable, et physiquement respectueux des limites du moteur à courant continu.